Category A Problems (The first three are Mandatory to solve by the mentor)

PROBLEM 1 • **Basic → Intermediate**

Bus Ticket Booking Validator

Problem Statement

An online bus booking platform ingests booking attempts from several channels, including a legacy agent-side tool that still builds ticket records with no validation at all — letting hollow, half-filled bookings slip into the day's manifest. Design BusTicket so an invalid booking can never be constructed in the first place, then write a batch processor that reads a list of raw {passengerName, destination} attempts and classifies each one as accepted, rejected, or a duplicate of an already-accepted pair. Decide for yourself exactly what makes a name or destination "meaningful" — the examples below will tell you if you got it right. A ticket, once marked as checked in, should not silently accept being checked in a second time.

Requirements

- *BusTicket must have no usable no-argument constructor — only a parameterized one.*
- *Invalid input must fail at construction time, not be discovered afterward.*
- *The same passengerName + destination pair must never be counted as a valid booking more than once, however many times it appears in the batch.*

Function Signature(s)

```
public BusTicket(String passengerName, String destination)
void markCheckedIn()
static void processBatch(String[][] rawBookings)
```

Examples

Input	Output
{{"Divya","Chennai"}, {"","Bangalore"}, {"Ravi123","Pune"}, {"Divya","Chennai"}, {" "," "}}	Valid: 1 | Rejected: 3 | Duplicates skipped: 1

Explanation: *Only the first Divya / Chennai pair is accepted. Work out precisely why each of the other four attempts fails — they don't all fail for the same reason.*

HINT

Blank, whitespace-only, and null aren't necessarily the same case to handle — and a name isn't invalid only when it's missing.

CONSTRAINTS

Batch size up to 1,000 entries.

Aim for a clean, correct classification before worrying about optimizing the duplicate check.

Key Concepts Tested

Parameterized constructors as a validation gate, string-validation logic, duplicate detection, idempotent state changes.

PROBLEM 2 • Intermediate

Remainder-Fair FareSplitter

Problem Statement

A group-travel booking feature needs a FareSplitter class that can be created three different ways — full details, total fare only, or nothing but a trip ID for a provisional split — without duplicating setup logic across constructors. The platform has also been burned by a real accounting bug before: naively dividing a fare across passengers and truncating each share causes the amounts actually collected to fall short of what's owed. Your fareBreakdown() must never lose money to rounding, no matter how unevenly the fare divides. You'll also need a way to tell whether a group is behind schedule on confirming their shares, and to reject a split that was never valid to begin with.

Requirements

- *The three constructors must be linked through this(...) chaining, not independent duplicated logic.*
- *A negative fare, or zero/negative passenger count, must be rejected at construction.*
- *fareBreakdown() must sum to exactly the original fare — test this against fares that don't divide evenly.*

Function Signature(s)

```
public FareSplitter(String tripId, double totalFare, int passengerCount)
public FareSplitter(String tripId, double totalFare)
public FareSplitter(String tripId)
double[] fareBreakdown()
boolean isConfirmationOverdue(int confirmed, int expected)
```

Examples

Input	Output
new FareSplitter("TRIP001", 100000, 3).fareBreakdown()	[33333.33, 33333.33, 33333.34]
new FareSplitter("TRIP003").fareBreakdown()	[0.0, 0.0]

Explanation: *100000 does not divide evenly by 3 — notice which share absorbs the extra paisa, and why it isn't the first one.*

Explanation: *A provisional, ID-only split should still produce a sensible, crash-free breakdown.*

HINT

If the fare doesn't divide evenly, the leftover has to go somewhere specific and consistent — not just “wherever.”

CONSTRAINTS

passengerCount will not exceed 60.

Results must be correct to 2 decimal places.

Key Concepts Tested

Constructor chaining combined with real validation, fair-division / remainder-distribution logic, defensive handling of degenerate inputs.

PROBLEM 3 • Intermediate

Bus Route Ranking Engine

Problem Statement

The route-planning team needs a way to rank a depot's bus routes for a priority-dispatch report, with ties — common among routes sharing the same priority level — broken the exact same way every single time, regardless of who regenerates the report. Design BusRoute's comparison logic and a ranking function precisely enough that two runs on the same input always agree. You'll need to encode a specific tie-breaking order (think about what a fair, deterministic ranking should check first, second, and last), and decide how two route codes that differ only in letter case should be treated — without ever altering how a route's own code is actually stored or displayed.

Requirements

- *BusRoute must resolve any field/parameter naming clash with this, and offer a second constructor that chains to the first with a sensible default priority value.*
- *The comparison method must return a signed result, following Java's own ordering convention — not a boolean.*
- *Implement the sort yourself; do not call a built-in sort utility.*

Function Signature(s)

```
public BusRoute(String routeCode, String routeName, int priority)
public BusRoute(String routeCode, String routeName)
int compareTo(BusRoute other)
static BusRoute[] rankRoutes(BusRoute[] routes)
```

Examples

Input	Output
[BusRoute("RT205L","Airport Express",3), BusRoute("rt201j","City Central",4), BusRoute("RT299T","Night Service")]	["rt201j", "RT205L", "RT299T"]

Explanation: *The third route was built with the 2-argument constructor, so its priority defaults. Work out where that places it in the ranking, and why the other two are ordered the way they are.*

HINT

What happens to two routes that are tied on every rule you check — does your sort still leave them exactly as they arrived?

CONSTRAINTS

Up to 200 routes in a single call.

An $O(n^2)$ sort is perfectly fine here — correctness of the ordering matters far more than speed.

Key Concepts Tested

this in every one of its forms, the Comparable ordering convention, multi-level tie-breaking, sort stability.

PROBLEM 4 • Intermediate → Advanced

Tiered Boarding Penalty Calculator

Problem Statement

The depot wants to replace its flat late-boarding penalty with a fairer, escalating system: the longer a passenger keeps the bus waiting at the gate, the higher the rate on the additional minutes — never retroactively on the whole waiting period. Rates: 0.5% of the ticket fare per minute for minutes 1–5, 1% per minute for minutes 6–15, and 2% per minute from minute 16 onward. Separately, the class's configured rate should now act as a minimum flat-fee floor, which only ever applies once a passenger is genuinely late — never when they board on time.

Requirements

- *The calculation rule, the field it depends on, and the class itself must each be locked against modification exactly where that protection makes sense.*
- *Negative ticketFare or minutesLate must be rejected at the point of calculation, not just at construction.*

Function Signature(s)

```
public BoardingPenaltyCalculator(double minimumPenaltyPercent)
final double calculatePenalty(double ticketFare, int minutesLate)
```

Examples

Input	Output
ticketFare = 1000, minutesLate = 0	Rs 0.0
ticketFare = 1000, minutesLate = 1	Rs 10.0
ticketFare = 1000, minutesLate = 16	Rs 145.0

Explanation: *minutesLate = 1 produces a smaller tiered amount than the 1% floor — work out which value actually gets charged, and why minutesLate = 0 must never trigger the floor at all.*

Explanation: *minutesLate = 16 crosses all three brackets — verify your bracket boundaries land exactly at minutes 5, 15, and 16.*

HINT

A passenger 6 minutes late is not “6 minutes at the minute-6-15 rate” — work out how many minutes actually fall in each bracket before multiplying anything.

CONSTRAINTS

minutesLate will not exceed 500.

ticketFare is a positive double up to roughly 10^5.

A closed-form O(1) calculation is preferred over a minute-by-minute loop.

Key Concepts Tested

final at the variable, method, and class level together; tiered/bracket calculation logic; boundary-safe arithmetic; floor logic with a special-case exception.

PROBLEM 5 • Advanced

Nightly Fleet Reconciliation Engine

Problem Statement

The depot's nightly reconciliation job processes every ticket account for the day's fleet — a mix of regular-coach and sleeper-coach bookings — and this year must also tolerate null placeholder entries left behind by a buggy upstream import, without ever letting the whole run crash because of one bad entry. Build a processor that safely handles a mix of real accounts, sleeper-coach accounts (which settle differently from everyone else), and outright nulls, computing a boarding penalty for every account actually processed and producing a trustworthy end-of-run summary. You're given three parallel arrays as input — decide for yourself what should happen if they don't all have matching lengths, and be ready to justify that choice the way a real depot manager would.

Requirements

- *BusTicketAccount must set up any one-time, class-level state using a static block, and support both a full constructor and a provisional one that chains to it.*
- *The processor must use instanceof to decide how each account is settled, and must never throw an uncaught exception because of a null entry.*
- *The penalty calculation must be final (reuse Problem 4's version if you built it, or a simpler flat-rate version otherwise — state which).*

Function Signature(s)

```
public BusTicketAccount(String bookingId, double ticketFare)
public BusTicketAccount(String bookingId)
final double calculatePenalty(int minutesLate)
void processAccount(BusTicketAccount account, double amount, int minutesLate)
static void processBatch(BusTicketAccount[] accounts, double[] amounts, int[] minutesLateArray)
```

Examples

Input	Output
accounts = {Sleeper("BK001",2000), null, BusTicketAccount("BK002",1200)} amounts = {1200, 900, 700} minutesLateArray = {10, 5, 0}	2 processed | 1 null skipped | 1 sleeper | 1 regular | grand total penalties = ...

Explanation: *Index 1 is null. Decide exactly what your processor does with it, and make sure the grand total only ever reflects accounts that were genuinely processed.*

HINT

What's worse for a depot: an error before anything runs, or a batch that quietly processes the wrong amount against the wrong passenger?

CONSTRAINTS

Batch size up to 10^4 accounts.

Aim for a single pass over the batch, with O(1) extra space beyond a handful of running counters.

Key Concepts Tested

Every concept from Problems 1–4 combined: constructor chaining, final, instanceof-based dispatch, static setup, null-safety, input validation, aggregation.